security/05/spec/models/user_spec.rb

```
require "rails_helper"

describe User do

  RSpec::Matchers.define :be_able_to_see do |*projects|
    match do |user|
      expect(user.visible_projects).to eq(projects)
      projects.all? { |p| expect(user.can_view?(p)).to be_truthy }
      (all_projects - projects).all? do |p|
        expect(user.can_view?(p)).to be_falsy
      end
    end
  end

  describe "visibility" do
    let(:user) { create(
      :user, email: "user@example.com", password: "password") }
    let(:project_1) { create(:project, name: "Project 1") }
    let(:project_2) { create(:project, name: "Project 2") }
    let(:all_projects) { [project_1, project_2] }

    it "a user can see their projects" do
      user.projects << project_1
      expect(user).to be_able_to_see(project_1)
    end

    it "an admin can see all projects" do
      user.admin = true
      expect(user).to be_able_to_see(project_1, project_2)
    end

    it "a user can see public projects" do
      user.projects << project_1
      project_2.update_attributes(public: true)
      expect(user).to be_able_to_see(project_1, project_2)
    end

    it "no dupes in project list" do
      user.projects << project_1
      project_1.update_attributes(public: true)
      expect(user).to be_able_to_see(project_1)
    end

  end

end
```

You've created a custom matcher, be_able_to_see, which takes in a list of projects and validates the can_view? and visible_projects in parallel, thereby asserting that the two methods stay in sync. It also validates that projects that aren't specified are not visible—you get around the fixture data by explicitly specifying the universe of all_projects. (In Minitest I'd use a custom assertion; you can see an example in the sample code at code/security/05/gatherer/test/models/user_test.rb.)